

Teoria per il Progetto 2 — Compressione DCT

Documento di studio: la teoria necessaria a capire il codice, spiegata in profondità, e poi come si collega all'implementazione. Niente dimostrazioni e niente calcoli lunghi: l'idea, il **perché**, l'intuizione visiva, e dove la ritrovi nel codice. La teoria viene prima; il collegamento al codice in fondo a ogni sezione.

0. Mappa concetti → codice

Concetto teorico	Dove nel codice	Funzione
Base ortonormale dei coseni	<code>dct_utils.py</code>	<code>compute_dct_matrix</code>
DCT 1D	<code>dct_utils.py</code>	<code>dct_1d_fast</code>
DCT 2D custom (matrice esplicita)	<code>dct_utils.py</code>	<code>dct_2d_homemade</code>
DCT 2D fast (FFT)	<code>dct_utils.py</code>	<code>dct_2d_fast</code>
IDCT 2D	<code>dct_utils.py</code>	<code>idct_2d_fast</code>
Maschera di taglio $k+l < d$	<code>dct_utils.py</code>	<code>frequency_keep_mask</code>
Pipeline compressione	<code>dct_utils.py</code>	<code>compress_image_array</code>
Caricamento BMP	<code>dct_utils.py</code>	<code>load_bmp_grayscale</code>
Benchmark tempi	<code>run_assignment.py</code>	<code>run_benchmark</code> , <code>time_function</code>
GUI	<code>gui.py</code>	<code>CompressionApp</code>

1. Il problema generale: rappresentare un segnale in una base

Intuizione di fondo. Un segnale (una riga di pixel, un'immagine) è un insieme di numeri. Possiamo descriverlo in molti modi equivalenti: come lista di valori "al naturale" (i pixel stessi), oppure come **somma di onde semplici** a frequenze crescenti. La seconda è la base delle trasformate.

Perché cambiare base serve a comprimere. Nella base "naturale" ogni coefficiente conta ugualmente: buttare un pixel qualunque fa un buco visibile. In una base di onde ben scelta, invece, **pochi coefficienti contengono quasi tutta l'informazione** e gli altri sono piccoli o trascurabili: posso buttarli e perdere poco. Questa è l'idea di tutta la compressione "trasformata": JPEG (DCT), MP3 (MDCT), ecc.

Cosa rende "buona" una base. Una base è buona per un certo tipo di segnali se:

1. è adatta alla struttura del segnale (le immagini sono "lisce a tratti");
2. i coefficienti **decadono** rapidamente: i primi grandi, gli altri piccoli;
3. è ortogonale/ortonormale, così cambiare base e tornare è esatto e facile.

La DCT è la base "buona" per le immagini naturali. Vediamo perché.

2. Da Fourier alla serie dei coseni: perché solo coseni

Serie di Fourier (caso continuo). Una funzione periodica di periodo p si scrive come somma di seni e coseni a frequenze k/p crescenti:

$$f(t) = a_0 + \sum [a_k \cos(\dots) + b_k \sin(\dots)]$$

Le componenti a_k sono i coseni (pari), le b_k i seni (dispari).

Il guaio della periodizzazione "bruta". Per applicare Fourier a una funzione definita solo su $[0, L]$, la si "periodizza" replicandola. Ma se la funzione ha valori diversi ai due estremi ($f(0) \neq f(L)$), la replicazione crea un **salto artificiale**. Un salto è una discontinuità $\rightarrow$ la serie di Fourier oscilla attorno al salto e non converge bene $\rightarrow$ **fenomeno di Gibbs** (ondulazioni che non si smorzano vicino al punto di salto).

Il trucco della DCT (periodizzazione con riflessione). Invece di replicare brutalmente, si **riflette** la funzione rispetto a un estremo, ottenendo una funzione **pari** di lunghezza $2L$, e poi la si periodizza di periodo $2L$. Conseguenze:

- la funzione estesa è **pari** $\rightarrow$ tutti i coefficienti dei **seni** (b_k) sono nulli (un seno è dispari, l'integrale pari-dispari su intervallo simmetrico è zero). Restano **solo i coseni** $\rightarrow$ "serie dei coseni".
- non si introduce alcun salto artificiale: la riflessione "incolla" i due estremi in modo continuo. Quindi **niente Gibbs artificiale** (resta solo il Gibbs delle discontinuità reali della funzione, che non possiamo evitare).

In una battuta: la DCT è la serie di Fourier della versione "riflessa e resa pari" del segnale. Questo la rende adatta ai segnali che non sono naturalmente periodici (come le immagini).

Collegamento al codice. Non implementiamo la riflessione: è la giustificazione teorica del perché usiamo solo coseni. Sia la matrice `compute_dct_matrix` sia `scipy.fft.dct(..., type=2)` usano già la base dei coseni derivata da questa costruzione. Il "type=2" di scipy è proprio la DCT-II, quella che nasce dalla riflessione ai punti medi $(2i+1)/(2N)$.

3. Il caso discreto: la funzione diventa un vettore

Idea. Un'immagine in scala di grigi è una griglia di pixel: non abbiamo una funzione ovunque, solo N valori campionati. La "funzione" diventa un **vettore** $f = (f_0, \dots, f_{N-1}) \in \mathbb{R}^N$.

Dove si campiona. I punti di campionamento sono scelti "a metà" di ogni sottointervallo:

$$t_i = (2i+1) / (2N), \quad i = 0..N-1$$

Questo "a metà" non è casuale: è esattamente la griglia che nasce dalla riflessione del paragrafo 2 (i punti medi degli intervalli dopo aver riflesso). Da qui deriva il $(2i+1)$ che compare ovunque nella DCT.

Base canonica vs base dei coseni. Un vettore di $\mathbb{R}^N$ si può scrivere su qualsiasi base di N vettori. La base canonica è $\{e_0, e_1, \dots\}$, dove e_k ha un 1 in posizione k e 0 altrove: ogni coefficiente è "un pixel". La base dei coseni è $\{w_0, w_1, \dots\}$, dove

$$(w_k)_i = \cos(k \cdot \pi \cdot (2i+1) / (2N))$$

- w_0 = vettore costante (tutti 1): la "frequenza zero", cioè il **valore medio** del segnale.
- w_1 = mezza oscillazione sul intervallo.
- w_2 = un'oscillazione intera.
- ... w_k = k mezze-oscillazioni → frequenza più alta.

Più k cresce, più la funzione di base oscilla velocemente = **alta frequenza**. Le basse frequenze (k piccolo) rappresentano le variazioni lente del segnale; le alte, i dettagli fini.

Ortogonalità. I vettori w_k sono ortogonali fra loro: il prodotto scalare $w_j \cdot w_k = 0$ se $j \neq k$. Le loro norme sono:

- $w_0 \cdot w_0 = N$ (è il vettore di tutti 1, norma² = N)
- $w_k \cdot w_k = N/2$ per $k > 0$

Dividendo ogni w_k per la sua norma si ottiene una base **ortonormale** (vettori ortogonali e di norma 1). Questo è importante: in una base ortonormale, il coefficiente è semplicemente il prodotto scalare, e l'inversa è la trasposta.

Collegamento al codice. I coefficienti di normalizzazione $\alpha_0 = 1/\sqrt{N}$ e $\alpha_k = \sqrt{2/N}$ sono esattamente $1/\sqrt{(w_k \cdot w_k)}$. La funzione `compute_dct_matrix` costruisce la matrice D le cui righe sono i w_k già normalizzati:

$$D[k,i] = \alpha_k \cdot \cos(k \cdot \pi \cdot (2i+1) / (2N))$$

D è la "versione matrice" della base ortonormale. Ogni riga = un vettore di base; ogni colonna = un punto di campionamento.

4. La DCT: cambiare base, vedere le frequenze

Cosa fa la DCT. Prende il vettore f (i pixel) e ne calcola i coefficienti a_k nella base dei coseni. Ogni a_k dice **quanto la frequenza k contribuisce** al segnale. Poiché la base è ortonormale:

$$a_k = f \cdot w_k \quad (\text{prodotto scalare con il vettore di base normalizzato})$$

In forma matrice: $a = D \cdot f$ (tutti i coefficienti in un colpo).

Interpretazione. Dopo la DCT ho una "mappa delle frequenze":

- a_0 = componente continua = $\sim$ valore medio $\times \sqrt{N}$.
- $a_1, a_2, \dots$ = quanto ogni frequenza è presente.
- Le a_k piccole = quella frequenza è poco importante → candidata al taglio.

La proprietà che fa funzionare la compressione (energy compaction). Per segnali "lisci a tratti" (come le immagini naturali), i coefficienti **decadono rapidamente**: $a_0, a_1, \dots$ sono grandi, e oltre un certo k sono quasi zero. Quindi l'energia del segnale è concentrata nelle **basse frequenze**. Posso buttarne la maggior parte e perdere poca energia → poca perdita visiva.

Quando invece non funziona. Se il segnale ha una **discontinuità reale** (bordo netto, scacchiera, rumore), i coefficienti **non decadono**: le alte frequenze servono davvero a ricostruire il salto. Se le taglio:

- il bordo si "smussa" e compare l'effetto di Gibbs (oscillazioni spurie attorno al bordo), perché sto troncando una serie di Fourier in corrispondenza di una discontinuità.

Collegamento al codice. `dct_1d_fast` fa esattamente $a = D \cdot f$ usando l'algoritmo veloce di scipy (`norm='ortho'` = la stessa nostra ortonormalità). La differenza "liscia vs discontinua" è ciò che vediamo negli esperimenti: il **gradiente** resta perfetto anche tagliando tanto, la **scacchiera** si rovina subito (PSNR crolla a ~10 dB a $d=2$).

5. La IDCT: tornare ai pixel

Cosa fa la IDCT. È l'inversa: dai coefficienti a_k ricostruisce i pixel f . Poiché la base è ortonormale, la matrice D è **ortogonale** e la sua inversa è la trasposta:

$$f = D^T \cdot a$$

Proprietà fondamentale: $IDCT(DCT(f)) = f$ esattamente (a meno di errori di macchina $\sim 1e-13$). La DCT non perde informazione di per sé: è solo un cambio di base reversibile. **L'informazione si perde solo quando taglio i coefficienti.**

Perché lo scaling è critico. Il round-trip torna esatto **solo se** DCT e IDCT usano la stessa normalizzazione. Se la DCT normalizza e la IDCT no (o viceversa), i fattori non si compensano e ottieni l'immagine riscalata o distorta. Per questo la traccia insiste tanto sullo scaling.

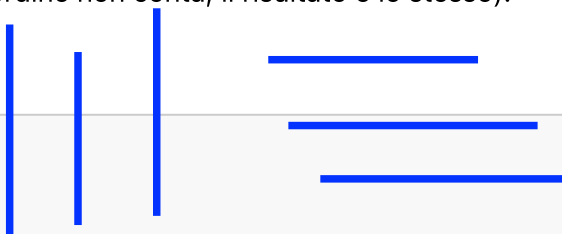
Collegamento al codice. `idct_2d_fast` usa `scipy.fft.idctn` con `norm='ortho'`, coerente con `dct_2d_fast`. Il test `test_idct_2d_reconstructs_original_matrix` verifica proprio che il round-trip restituisca l'originale (errore $\sim 1e-10$).

6. DCT 2D: separabilità (righe poi colonne)

Idea. Un'immagine è una matrice `fxH` non un vettore. La DCT si estende al 2D sfruttando una proprietà del coseno: il **nucleo è separabile**, cioè la trasformata 2D si fa applicando la 1D **prima a tutte le colonne, poi a tutte le righe** (o viceversa: l'ordine non conta, il risultato è lo stesso).

In forma matrice:

$$c = D \cdot f \cdot D^T$$



- $D \cdot f$ applica la DCT 1D a ogni colonna (trasforma "in verticale").
- D^T applica la DCT 1D a ogni riga del risultato (trasforma "in orizzontale").

L'inversa è $f = D^T \cdot c \cdot D$.

Interpretazione del coefficiente $c_{\{k, \ell\}}$. Indica quanto è presente la componente che oscilla k volte in verticale e ℓ volte in orizzontale. Quindi:

- $c_{\{0, 0\}}$ = componente continua = **media del blocco** (luminosità media).
- $c_{\{k, 0\}}$ = variazione solo verticale, frequenza k .
- $c_{\{0, \ell\}}$ = variazione solo orizzontale, frequenza ℓ .
- $c_{\{k, \ell\}}$ = variazione combinata, "frequenza totale" $\sim k + \ell$.

Le basse frequenze (k, ℓ piccoli) stanno in **alto a sinistra**, le alte in **basso a destra**. Questa disposizione è alla base della maschera di taglio.

Perché la separabilità è un vantaggio. Senza separabilità, la DCT 2D costerebbe come una trasformata 2D generica. Con la separabilità si riduce a "due DCT 1D", che si possono fare in modo efficiente (e la fast le fa via FFT).

Collegamento al codice.

- **Custom** (`dct_2d_homemade`): fa letteralmente $D @ f @ D.T$ con numpy. Due prodotti matrice-matrice $\rightarrow$ costo **$O(N^3)$** .
- **Fast** (`dct_2d_fast`): `scipy.fft.dctn(f, norm='ortho')` fa la stessa cosa ma via FFT $\rightarrow$ costo **$O(N^2 \cdot \log N)$** . Il test `test_homemade_dct_2d_matches_fast_dct_2d` verifica che danno lo stesso risultato (errore $\sim 1e-10$): stessa trasformata, cambia solo la velocità.

7. Lo scaling ortonormale (la cosa su cui la traccia insiste)

Il problema delle convenzioni. Esistono diverse "DCT": la DCT-I, DCT-II, DCT-III, DCT-IV, e per ciascuna si può scegliere se normalizzare o meno e come. Scelte diverse:

- cambiano i valori numerici dei coefficienti di un fattore costante;
- possono far sì che DCT e IDCT non si compenso;
- fanno fallire il confronto con i valori di riferimento della traccia.

Noi usiamo la DCT-II ortonormale:

$$\alpha_0 = 1/\sqrt{N}, \quad \alpha_k = \sqrt{(2/N)} \quad \text{per } k > 0$$

Con questa scelta:

- D è **ortogonale** ($D^{-1} = D^T$): IDCT = DCT con la trasposta, niente fattori extra da ricordare.
- L'energia si conserva: $\|a\|^2 = \|f\|^2$ (Parseval), utile per ragionare su quanta energia taglio.
- Corrisponde in scipy a `norm='ortho'`.

Collegamento al codice. Entrambe le implementazioni usano questa convenzione:

- `compute_dct_matrix`: mette $\alpha[0] = 1/\sqrt{N}$ e $\alpha[k] = \sqrt{2/N}$.
- `dct_1d_fast`, `dct_2d_fast`, `idct_2d_fast`: tutte con `norm='ortho'`. I casi test della traccia (riga 1×8 e blocco 8×8) servono a verificare che lo scaling è giusto: i nostri risultati coincidono con la tabella della traccia a meno dell'arrotondamento a 3 cifre.

8. La maschera di taglio: dove si "comprime"

L'idea. Dopo la DCT ho i coefficienti $c_{\{k, \ell\}}$. Le basse frequenze (k, ℓ piccoli, in alto a sinistra) contengono quasi tutta l'energia; le alte (in basso a destra) sono spesso trascurabili. **Le taglio.**

Come si taglia nel progetto. Si conserva un coefficiente $c_{\{k, \ell\}}$ se

$$k + \ell < d$$

cioè si tengono i coefficienti **sotto l'antidiagonale** di livello d , e si azzerano quelli sopra o su di essa ($k + \ell \geq d$).

Perché $k + \ell$ (la "frequenza totale"). $k + \ell$ è una misura semplice di "frequenza combinata" (verticale + orizzontale). Tagliare per $k + \ell$ conserva un **triangolo di basse frequenze** in alto a sinistra. È una scelta:

- **semplice** da spiegare e da implementare;
- **vettorizzabile** con una sola espressione booleana;
- **coerente** con l'idea che le frequenze "totalmente alte" siano le meno importanti.

Notare che non è l'unica scelta possibile (il vero JPEG usa una scansione a zig-zag + una matrice di quantizzazione, più sofisticata), ma cattura l'idea essenziale.

I due estremi:

- $d = 0 \rightarrow$ azzerare **tutto**. La IDCT di un solo coefficiente nullo... anzi di tutti zero dà il valore medio (zero, ma poi saturato). In pratica l'immagine diventa **grigio uniforme** = media del blocco.
- $d = 2F - 2 \rightarrow$ tengo **tutto tranne** $c_{\{F-1, F-1\}}$ (l'angolo in basso a destra = la singola frequenza massima). Taglio minimo possibile.

In mezzo: d piccolo = taglio aggressivo (molta compressione, poca qualità, Gibbs sui bordi); d grande = taglio lieve (poca compressione, alta qualità).

Collegamento al codice. `frequency_keep_mask(F, d)` costruisce una matrice booleana:

```
rows, cols = np.indices((F, F))
mask = rows + cols < d      # True = tenuto, False = azzerato
```

Poi in `compress_image_array`: `coefficients[~mask] = 0.0`. Il test `test_frequency_mask_keeps_only_coefficients_before_diagonal` verifica la forma della maschera su $F=4$, e i casi estremi $d=0$ (tutto False) e $d=6$ (15 tenuti su 16).

9. La pipeline di compressione, passo per passo

Per ogni blocco f di dimensione $F \times F$:

1. **DCT2**: $c = \text{DCT2}(f)$ — passo nel dominio delle frequenze.
2. **Taglio**: azzero i coefficienti con $k+l \geq d$ usando la maschera.
3. **IDCT2**: $ff = \text{IDCT2}(c)$ — torno ai pixel (approssimati, ho perso le frequenze tagliate).
4. **Arrotondamento + saturazione**: `np rint` (intero più vicino) e `np.clip(0, 255)` — i pixel devono essere byte validi (0-255).
5. **Ricomposizione**: rimetto il blocco al suo posto nell'immagine.

Prima di tutto: l'immagine viene divisa in blocchi $F \times F$ partendo in alto a sinistra; gli avanzi (righe/colonne non multiple di F) **si scartano**, come dice la traccia.

Dove si perde informazione (irreversibilità). Due punti:

- il **taglio delle frequenze** (è la compressione vera e propria);
- l'**arrotondamento + saturazione** dei pixel ricostruiti a interi 0-255. Anche se non tagliassi nulla ($d=2F-2$), l'arrotondamento introduce un piccolo errore. Per questo a d massimo il PSNR è altissimo ma non sempre ∞ .

Perché si lavora a blocchi e non sull'immagine intera. Due motivi:

- **località**: le immagini sono lisce "a tratti"; un blocco piccolo è più probabile che sia uniforme $\rightarrow$ i coefficienti decadono meglio $\rightarrow$ compressione efficace. Su un'immagine intera ci sarebbe di tutto (bordi ovunque) e i coefficienti non decaderebbero.
- **costo**: la DCT 2D su un blocco $F \times F$ costa $O(F^2 \log F)$; su $H \times W$ intera costerebbe $O(HW \log(HW))$, molto di più. E con blocchi piccoli si può parallelizzare/confina gli artefatti.

Collegamento al codice. Tutto questo è in `compress_image_array`:

- `cropped_height = (height // F) * F` $\rightarrow$ scarta gli avanzi.
- doppio ciclo `for top ... for left ...` $\rightarrow$ scorre i blocchi.
- dentro il ciclo: `dct_2d_fast` $\rightarrow$ `coefficients[~mask]=0` $\rightarrow$ `idct_2d_fast`.
- alla fine: `np.clip(np rint(compressed), 0, 255).astype(np.uint8)`.

Scelta d'implementazione: la maschera si calcola **una volta sola** fuori dai cicli (è uguale per tutti i blocchi, dipende solo da F e d). È un piccolo ma importante ottimizzazione.

10. Metriche di qualità: MSE e PSNR

MSE (Mean Squared Error): media dei quadrati delle differenze tra originale e compressa. Più basso = meglio. Zero = ricostruzione identica.

PSNR (Peak Signal-to-Noise Ratio), in dB:

$$\text{PSNR} = 20 \cdot \log_{10} \left(\frac{255}{\sqrt{\text{MSE}}} \right)$$

- 255 = valore massimo di un pixel in scala di grigi (il "picco" del segnale).
- $\sqrt{\text{MSE}}$ = rumore quadratico medio (la "deviazione").
- Il rapporto picco/rumore, in scala logaritmica (dB).
- Più MSE è piccolo $\rightarrow$ PSNR più alto $\rightarrow$ migliore la qualità.
- $\text{MSE} = 0 \rightarrow \text{PSNR} = \infty$ (ricostruzione perfetta).

Perché il logaritmo e i dB. Perché l'occhio percepisce la qualità in modo logaritmico: un raddoppio del rumore non si "sente" come il doppio. I dB rendono la metrica più correlata alla percezione. Inoltre il PSNR comprime una grande gamma di qualità in numeri gestibili (10-100 dB).

Cosa significano i numeri in pratica:

- PSNR > 40 dB: qualità eccellente, indistinguibile dall'originale.
- PSNR 30-40 dB: buona.
- PSNR 20-30 dB: accettabile ma con difetti visibili.
- PSNR < 20 dB: difetti forti (Gibbs evidente, blocchi visibili).

PSNR come funzione di d. Cresce monotonamente con d (taglio meno = qualità maggiore). Per $d \rightarrow 2F-2$ tende al massimo consentito dall'arrotondamento. La pendenza dipende dall'immagine: ripida per immagini lisce (pochi coefficienti bastano), piatta per immagini con bordi netti (anche tagliando poco si guadagna poco finché non si recuperano le alte frequenze).

Collegamento al codice. In `run_assignment.py`, `calculate_metrics` calcola MSE e PSNR; i valori vengono mostrati nei titoli delle griglie e salvati nei CSV. Negli esperimenti: il gradiente arriva a ∞ ($d=14$), le foto reali superano 60 dB a $d=14$, le scacchiere crollano a ~ 10 dB a $d=2$.

11. Complessità: custom vs fast

Custom (matrice esplicita): $D \cdot f \cdot D^T$ = due prodotti matrice-matrice. Ogni prodotto matrice-matrice su $N \times N$ costa $O(N^3)$ (N^2 elementi, ognuno prodotto di N termini). Totale $O(N^3)$. Conseguenza: raddoppiando N , il tempo dovrebbe $\sim$ ottuplicarsi ($\times 8$) — il "fattore 8" è il marchio di una scalatura cubica.

Fast (FFT-based): scipy usa `pocketfft`, che calcola la DCT via FFT su un vettore di lunghezza $2N$. Custo 1D: $O(N \cdot \log N)$. Custo 2D (righe + colonne, per separabilità): $O(N^2 \cdot \log N)$. Molto più lento a crescere del N^3 .

Cosa si vede nel benchmark:

- Per N piccoli (≤ 32) i tempi sono quasi uguali e quasi costanti: domina l'**overhead** delle chiamate di funzione, dell'allocazione, del setup. La scalatura asintotica non si vede ancora.
- Da $N \approx 64$ in poi si vede la differenza: la curva custom sale molto più ripida (N^3) di quella fast ($N^2 \cdot \log N$).
- La fast mostra **piccole irregolarità**: la FFT è più efficiente su N che si fattorizzano bene (potenze di 2), meno su N "scomodi". Per questo i tempi fast non sono una curva perfettamente liscia.

Confronto pratico. A N grande la fast è nettamente più veloce (nell'ordine di qualche a $N=1024$, e il divario cresce con N). Nella compressione, dove si processano migliaia di blocchi, si usa ovviamente la fast.

Collegamento al codice. `run_benchmark` cronometra entrambe su matrici casuali $N \times N$ con $N = 8, 16, \dots, 1024$. `time_function` prende il **tempo minimo** su 3 ripetizioni (il minimo è meno rumoroso della media, perché esclude i "scambi" del sistema operativo). Il grafico è in scala **semilog-y** (solo le ordinate logaritmiche), come chiede la traccia: così le curve di potenza (N^3 , $N^2 \log N$) diventano rette e si confrontano a vista.

12. Le immagini: perché certe si comprimono bene e altre no

Tipo di immagine	Comportamento DCT	Risultato
Gradiente (liscio)	coeff. decadono subito	compressione ottima, PSNR alto anche tagliando tanto
Foto reale (localmente liscia)	coeff. decadono bene entro ogni blocco	buona compressione, PSNR 60+ dB a $d=14$
Scacchiera (bordi netti)	coeff. NON decadono	Gibbs forte, PSNR crolla a d piccolo
Lettera "C" (contorno curvo)	bordo curvo = discontinuità	perde nitidezza, Gibbs sul contorno

Il principio unificante. Tutto dipende da **quanto l'immagine è liscia entro un blocco $F \times F$** . Più è liscia, più i coefficienti DCT decadono, più posso tagliare. I bordi netti sono il nemico della DCT, perché un bordo è una discontinuità e le discontinuità richiedono alte frequenze per essere rappresentate.

Effetto della dimensione del blocco F .

- **F grande** (es. 32): un blocco copre più contesto, quindi "capisce" meglio l'andamento generale → in teoria meglio. Ma se taglio male, l'artefatto copre aree grandi → **effetto a blocchi** molto visibile. Inoltre costa $O(F^2 \log F)$ per blocco.
- **F piccolo** (es. 4): più veloce, ma il blocco è troppo piccolo per "catturare" variazioni significative → la DCT concentra meno l'energia → compressione meno efficace.
- **F = 8**: il compromesso JPEG standard. Buona concentrazione di energia, artefatti confinati in aree piccole, costo contenuto.

L'effetto a blocchi (blocking artifact). Poiché ogni blocco è processato indipendentemente, i bordi fra blocchi possono non combaciare perfettamente dopo la ricostruzione, specialmente a d piccolo. Si vede come una griglia fantasma sull'immagine. È l'altro artefatto tipico del JPEG, oltre al Gibbs.

Invarianza della scacchiera a $F=8$. Le scacchiere 20×20 , 40×40 , ..., 640×640 danno PSNR identici a $F=8$. Perché? Il pattern ha celle di lato divisibile per 8, quindi **ogni blocco 8×8 è identico** → stessa DCT → stesso taglio → stesso PSNR. A $F=16$ e $F=32$ il rapporto periodo/celle cambia, e i valori differiscono leggermente.

Collegamento al codice. Tutto questo si osserva nei CSV `results/examples/*_metrics.csv` e nelle griglie generate da `run_assignment.py`. La tabella `tab:checker` nella relazione mostra esplicitamente l'invarianza a $F=8$.

13. Rapporto con il JPEG vero

Il progetto è "JPEG-style ma senza matrice di quantizzazione". Le differenze con il JPEG reale:

Aspetto	Questo progetto	JPEG reale
Trasformata	DCT2 ortonormale, blocchi F×F	DCT2, blocchi 8×8 fissi
Taglio	maschera binaria $k+l < d$ (si/no)	matrice di quantizzazione: divide ogni coeff. per un valore che cresce con la frequenza → taglio "soft" e differenziato
Codifica dei coeff.	nessuna (si tiene/azzerà)	codifica entropica (Huffman/run-length) dopo la quantizzazione
Colore	solo grigio	YCbCr, sottocampionamento cromatico
Arrotondamento	$\text{rint} + \text{clip } [0,255]$	simile

Cosa manca di "speciale". La matrice di quantizzazione del JPEG è più furba di una maschera binaria: invece di "tenere tutto o nulla", **divide** ogni coefficiente per un fattore che cresce con la frequenza (le alte frequenze vengono divise per numeri grandi → dopo l'arrotondamento diventano spesso zero, ma non sempre). Questo dà un taglio più fine e adattivo. Noi, per semplicità didattica, usiamo un taglio netto (maschera booleana), come chiede la traccia. L'idea di fondo (concentrare l'energia nelle basse frequenze e tagliare le alte) è però la stessa.

Collegamento al codice. `frequency_keep_mask` implementa il taglio binario; `compress_image_array` non ha nessuna matrice di quantizzazione né codifica entropica. È la versione "minimum viable" della compressione DCT.

14. Design choices chiave del codice (domande tipiche d'esame)

D: Perché la maschera si calcola una volta sola? R: Dipende solo da F e d, non dal contenuto del blocco. Calcolarla fuori dal ciclo evita di rifare lo stesso lavoro per ogni blocco: è un'ottimizzazione gratis dato che la maschera è uguale per tutti i blocchi.

D: Perché si scartano gli avanzati invece di fare padding? R: La traccia lo richiede esplicitamente ("scartando gli avanzati"). Inoltre è più semplice e deterministico: niente scelta di come paddare (zero? replica? riflessione?), niente bordi artefatti. La GUI mostra la dimensione effettivamente usata, così l'utente sa cosa ha ottenuto.

D: Perché usare la fast e non la custom nella compressione? R: La custom è didattica (prima parte). Nella compressione si processano migliaia di blocchi: la fast è $O(N^2 \cdot \log N)$ vs $O(N^3)$, molto più veloce. Danno lo stesso risultato (verificato dai test), quindi scegliere la fast è sempre meglio in produzione.

D: Perché `norm='ortho'`? R: È la convenzione ortonormale della traccia. Senza, la matrice D non sarebbe ortogonale, la IDCT non sarebbe la trasposta, il round-trip non tornerebbe esatto, e i casi test numerici non coinciderebbero con la tabella della traccia.

D: Perché `np rint` + `np clip`? R: La IDCT restituisce reali; i pixel sono interi in $[0, 255]$. Arrotondare (all'intero più vicino) e saturare (valori fuori range $\rightarrow 0$ o 255) converte in byte validi. È anche una fonte di errore irreversibile, oltre al taglio.

D: Cosa cambia tra `d piccolo` e `d grande`? R: `d piccolo` = taglio aggressivo = tengo pochi coefficienti (basse frequenze solo) = molta compressione ma poca qualità (e Gibbs sui bordi netti). `d grande` = taglio lieve = tengo quasi tutto = poca compressione, alta qualità.

D: Cos'è il fenomeno di Gibbs qui? R: Quando taglio le alte frequenze vicino a un bordo netto, la ricostruzione oscilla attorno al bordo invece di essere netta. Succede perché tronco la serie di Fourier (qui, di coseni) in corrispondenza di una discontinuità: le alte frequenze servivano a "costruire" il salto, e togliendole il salto diventa un'onda smussata con overshoot.

D: Perché la DCT e non la DFT (con i complessi)? R: Tre motivi. (1) La DCT lavora con reali (coseni), niente parte immaginaria: metà memoria e calcoli. (2) Per immagini reali concentra meglio l'energia nelle basse frequenze rispetto alla DFT. (3) Evita l'artefatto della periodizzazione "bruta" (la DFT assume periodicità esatta, la DCT no, grazie alla riflessione), quindi meno Gibbs artificiale.

D: Perché blocco 8×8 e non 16×16 o 4×4 ? R: È un compromesso. 8×8 è abbastanza grande da catturare variazioni significative (buona concentrazione di energia) ma abbastanza piccolo da confinare gli artefatti e tenere il costo basso. È lo standard JPEG.

D: Cosa succede con `d=0`? R: La maschera è tutta False $\rightarrow$ azzerò tutti i coefficienti. La IDCT di una matrice di zeri è zero, ma dopo clip/arrotondamento... in realtà il blocco diventa nero (0) o, se si tenesse solo `c_{0,0}`, il valore medio. Con `d=0` l'immagine collassa al valore medio di ogni blocco $\rightarrow$ grigio uniforme a blocchi.

D: Il PSNR può essere infinito? R: Sì, quando $MSE=0$, cioè la ricostruzione è identica all'originale (in floating point). Succede per immagini molto lisce a `d` alto: le frequenze tagliate erano già praticamente zero, quindi non si perde nulla. Il gradiente a `d=14` dà $PSNR=\infty$.

15. Come si usa il software (da sapere all'esame)

- **GUI** (`gui.py`): scegli .bmp, inserisci `F` e `d`, premi Comprimi. Mostra originale e compressa affiancate + suggerimenti live sui valori ammessi (min `F` dato `d`, range di `d` dato `F`, max `F` data l'immagine).
- **Benchmark** (`run_assignment.py`): genera il grafico dei tempi e le griglie di esempio. Flag: `--size` (`N`, ripetibile), `--repeats`, `--image`, `--block-size`, `--cutoff`, `--no-examples`.
- **Test** (`test_assignment.py`): valida i casi test della traccia, il round-trip, la maschera, la compressione. Si gira con `python test_assignment.py`.

All'esame il docente può chiedere di eseguire il software su immagini sue: la GUI è la via più rapida, ma conviene sapere anche la riga di comando.

Riassunto in una frase per concetto

1. **Cambio base:** descrivo il segnale come somma di onde invece che come pixel, perché poche onde bastano $\rightarrow$ comprimibile.

2. **Fourier→coseni**: riflettere la funzione la rende pari → servono solo coseni, niente Gibbs artificiale.
3. **DCT**: cambio base dai pixel ai coefficienti di coseni ortonormali. Liscia = coeff. che decadono; discontinua = no.
4. **IDCT**: il ritorno; inversa = trasposta perché la base è ortonormale.
5. **2D**: separabile, righe poi colonne, $c = D \cdot f \cdot D^T$.
6. **Scaling**: ortonormale, `norm='ortho'`, altrimenti non torna il round-trip.
7. **Maschera $k+l < d$** : tengo le basse frequenze (triangolo in alto a sx), taglio le alte.
8. **Pipeline**: DCT2 → taglio → IDCT2 → rint+clip → ricomponi, blocco per blocco, avanzi scartati.
9. **Custom $O(N^3)$ vs fast $O(N^2 \log N)$** : stessa matematica, velocità diversa.
10. **PSNR**: misura di qualità in dB, ∞ se perfetto; >40 eccellente, <20 pessimo.
11. **Gibbs + blocking**: i due artefatti tipici: oscillazioni sui bordi netti e griglia fra blocchi, peggiorano a d piccolo e F grande.
12. **JPEG vero**: stessa idea, ma con matrice di quantizzazione (taglio soft) e codifica entropica; noi facciamo la versione binaria didattica.